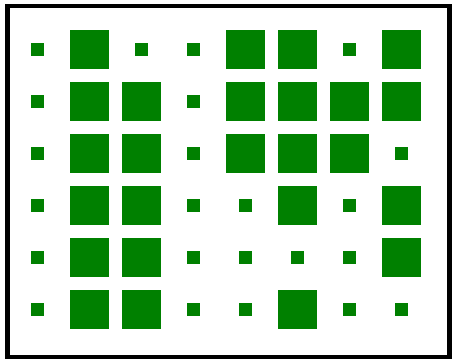


Class 5
Lexical Analysis (Scanning)

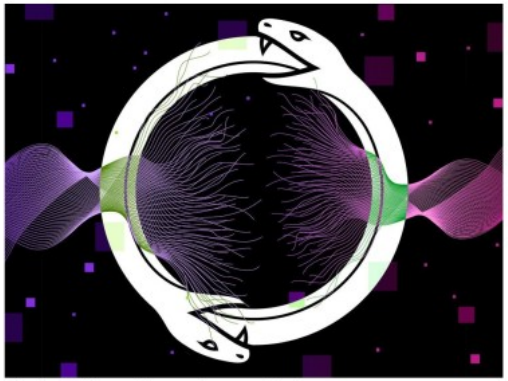
CS 330
Programming Languages and Compilers

AI Coding Assistants Are Getting Worse

Newer models are more prone to silent but deadly failure modes

Jamie Twiss
08 Jan 2026
https://spectrum.ieee.org/ai-coding-degrades?utm_source=ai-report.kdnuggets.com&utm_medium=newsletter&utm_campaign=ai-coding-taking-longer

Jamie Twiss is a banker and a data scientist who works at the intersection of data science, artificial intelligence, and consumer lending.



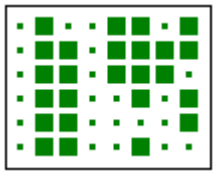
Nicole Millman; Source images: iStock

In recent months, I've noticed a troubling trend with AI coding assistants. After two years of steady improvements, over the course of 2025, most of the core models reached a quality plateau, and more recently, seem to be in decline. A task that might have taken five hours assisted by AI, and perhaps 10 hours without it, is now more commonly taking seven or eight hours, or even longer. It's reached the point where I am sometimes going back and using older versions of large language models (LLMs).

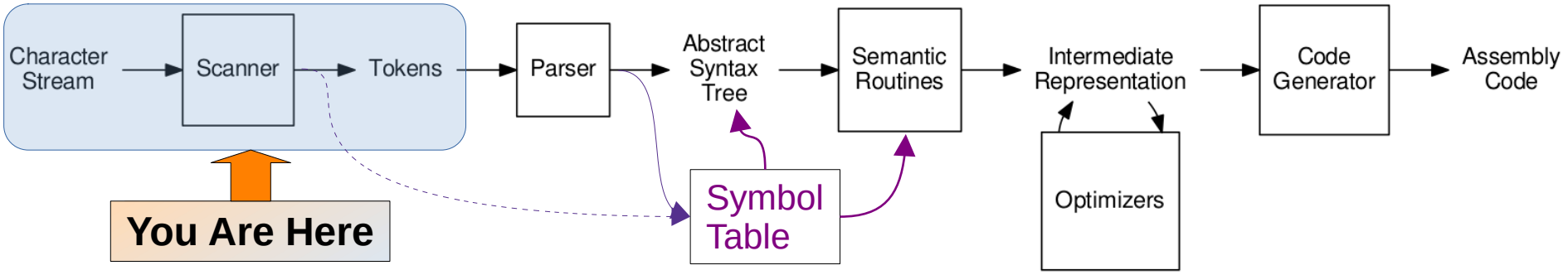
I use LLM-generated code extensively in my role as CEO of Carrington Labs, a provider of predictive-analytics risk models for lenders. My team has a sandbox where we create, deploy, and run AI-generated code without a human in the loop. We use them to extract useful features for model construction, a natural-selection approach to feature development. This gives me a unique vantage point from which to evaluate coding assistants' performance.

LLM Roundtable

Compilation Process Overview

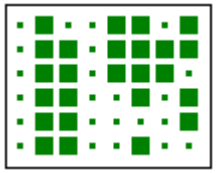


Thain, D. (2020). Introduction to compilers and language design.



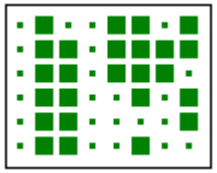
- **Scanner (Lexical analyzer) – tokenizes**
- Parser (Syntax analyzer) – lexical units syntax check
  - Creates symbol table and AST
- Semantic Routines – static semantics analysis/check
- Intermediate Rep – lower level language (optional)
- Optimizer – performance improvements (optional)
- Code Generator – outputs assembly lang (in our case)

Syntax



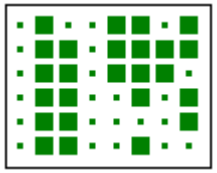
- A language is defined by a set of syntax rules
  - Groupings of characters that are defined for the language
  - Permitted ordering of the groups

Statements, Lexemes, Tokens…



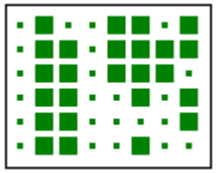
- A **lexeme** is a low-level syntactic unit such as a literal, variable, operator, or keyword (*specific*)
- A **token** is a categorization for lexemes (*general*)
  - Provides a grouping for lexemes.
    - Each lexeme must be categorized as a token
  - token:lexeme is a 1:many relationship, including 1:1
- A **statement** is a group of lexemes, while *syntax is defined as a group of tokens (and higher-level abstractions)*

Common Token Types



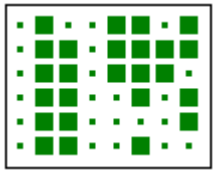
- Keywords (may be reserved words as well)
- Identifiers
  - *Names*: variables, functions, classes
- Operators
- Literal values
  - Numbers (integer, float, base, scientific notation)
  - Characters
  - Strings
- Non-tokenized: (no associated token)
  - Unimportant indentation, spacing around operators, blank lines, comments

Lexical Analysis Overview



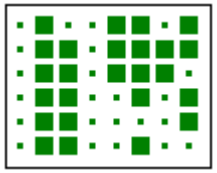
- **Lexical analyzer** finds patterns in the source code
  - a.k.a. scanner
- **Patterns** are defined to **match** the language's **lexemes**
  - Often **defined as REs** and **implemented as FAs**
  - **Associated** with **tokens**
- When a lexeme is matched by a pattern, the lexical analyzer emits the associated token

Tokens



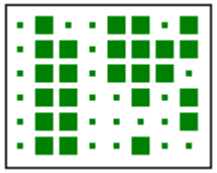
- Tokens represent groupings of lexemes by concept in the language (variable, type...)
- Tokens are represented as lookup values (constants or enumerations)

Lexical Analysis Output



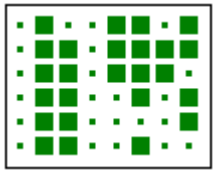
- The result of lexical analysis is a stream of tokens and associated metadata (such as user-defined terms and literals in a symbol table)
  - The input program is converted to its concepts, represented as tokens
  - This means that every concept in a language must have an associated token and at least one lexeme

Ambiguity Is Problematic



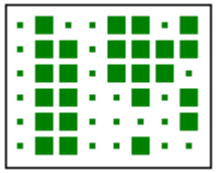
- If 2 or more lexemes start with the same character(s) there will be ambiguity until a differentiating character is encountered
  - In a simple case, the ambiguity exists for a single character and involves whether that character ends a lexeme or the lexeme includes the next character(s)
  - If the next character is part of another lexeme then it must be put back at the front of the processing queue and the existing lexeme mapped to its token

Ambiguity Is (Somewhat) Necessary



- It is very difficult to create a syntax where such ambiguities do not exist
  - To do so reduces readability, writeability
- Generally the scanner is forced to deal with tracking multiple matching patterns until they are ruled out by subsequent characters
  - Although not conceptually challenging, it requires redundant coding

Lexical Analysis Example



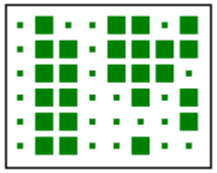
- Given the statement

```
result = oldsum – value / 100;
```

- The lexemes and tokens identified by the lexical analyzer might be:

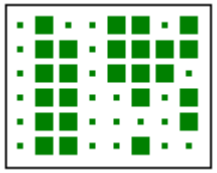
Token	*Lexeme*
IDENT	result
ASSIGN_OP	=
IDENT	oldsum
SUB_OP	-
IDENT	value
DIV_OP	/
INT_LIT	100
SEMICOLON	;

Lexical Analyzer Operations 1/2



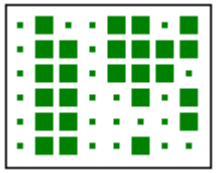
- As the lexical analyzer identifies each token, it is made available to the parser (syntax analyzer)
  - **Parser accepts tokens one at a time**
    - It does not see the source code
- Lexical analyzer skips past comments and unimportant whitespace
  - No tokens for these concepts – hence not seen by the parser
  - Reduces parser's view to logic only

Lexical Analyzer Operations 2/2



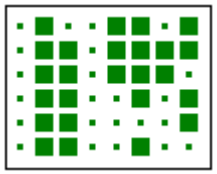
- The lexical analyzer may add user-defined names to a symbol table (variables, methods, classes…) which will be used by the parser and later steps in the process
- Few syntax errors can be found at this stage
  - Doesn't match any lexeme definition

Implementing a Lexical Analyzer

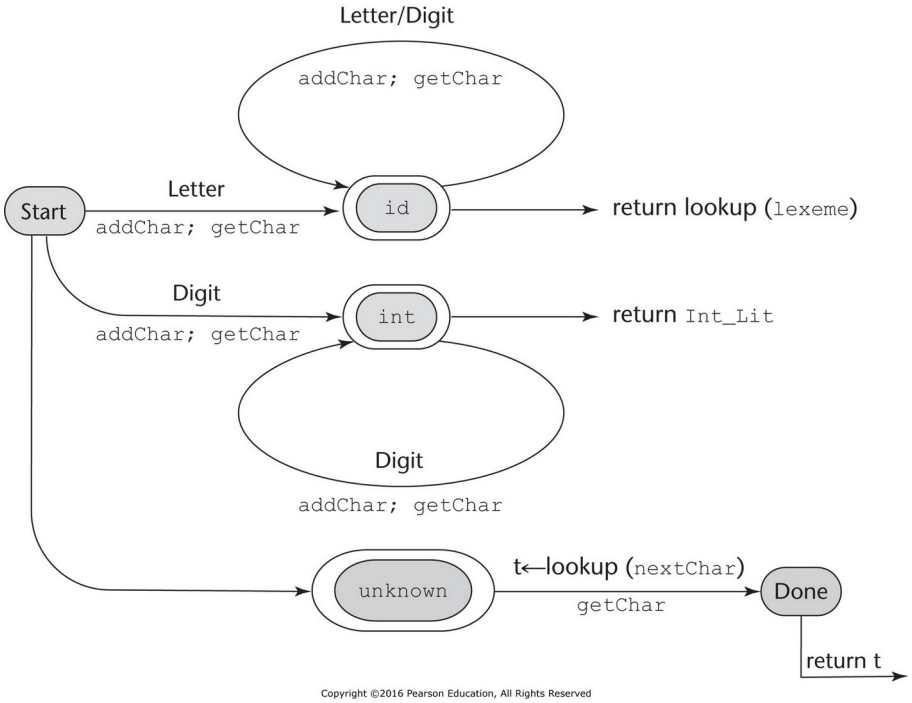


- A couple of ways to implement
  - Describe the definitions of token patterns using a formal language, such as regular expressions (discussed later), and write a program to match those
  - Create a state transition diagram for each token pattern and write a program which implements the diagrams

State Diagram Introduction

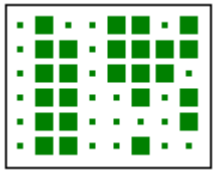


- A state diagram is a directed graph
  - Nodes are labeled with state names
  - Edges are labeled with input characters that cause transitions between states



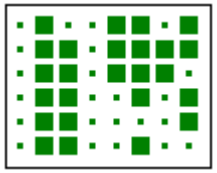
Copyright ©2016 Pearson Education, All Rights Reserved

State Diagram Definition



- Formally these describe **finite automata**
  - a.k.a. finite-state machines
  - Will be in one (or more?) of a finite number of states at any time
  - Allow for recognition of **regular languages**
  - **Regular grammars** generate regular languages
  - We'll explore grammars in a couple of weeks
- *The tokens of a programming language are a regular language; a lexical analyzer is a finite automaton*

Finite Automaton Definition



- A finite automaton is a 5-tuple (quintuple)

  $(\Sigma, S, s0, \delta, F)$

- where

  Σ (Sigma) is a finite set of input symbols

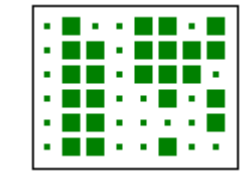
  S is a finite set of states

  $s0$ (the start state) is an element of S

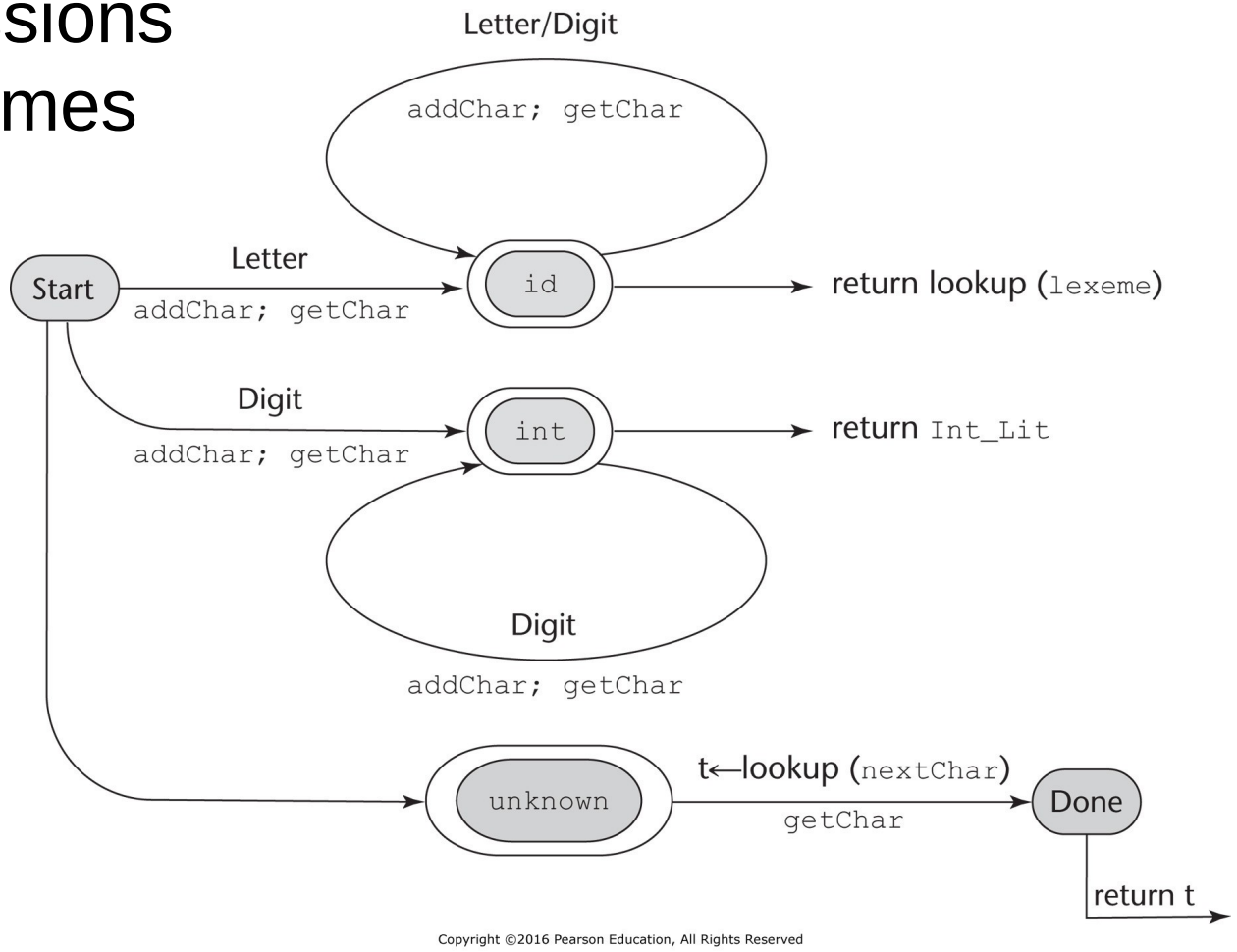
  δ (delta, the transition function) is a state-transition function from $S \times \Sigma \rightarrow S$

  F (the final or accepting states) is a subset of S

Example State Diagram

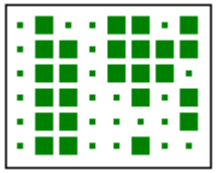


- Defining syntax for arithmetic expressions using variable names and integer literals as operands
- Here is the state diagram for parsing the input



Copyright ©2016 Pearson Education, All Rights Reserved

State Diagram as Finite Automaton



- Preceding diagram defines FA using state diagram
- Input symbols specified along the arrows

  `Σ = { Letter, Digit, (everything else) }`
- States represented by ovals

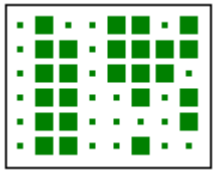
  `S = { Start, id, int, unknown }`
- Arrows are transitions among states

  δ can be drawn with the headings:

  `From-state, Input symbol, To-state`
- The start state is `Start (s0 = Start)`
- Final (accepting) states are drawn with two ovals

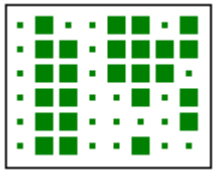
  `F = {id, int, unknown}`

Reading the State Diagram



- Begin at **Start** state; depending on the **input symbol** read (Letter, Digit, other), move to the appropriate state. Process ends when:
  - An input symbol is read while in a state that isn't on an edge leaving that state
  - There is no more input
- However, if in **Start** state and **input** is other than a Letter or a Digit, move to the **unknown state** and immediately perform a **lookup** (no arrow to remain in accepting state) and **return**

Identifying Lexemes as an FA

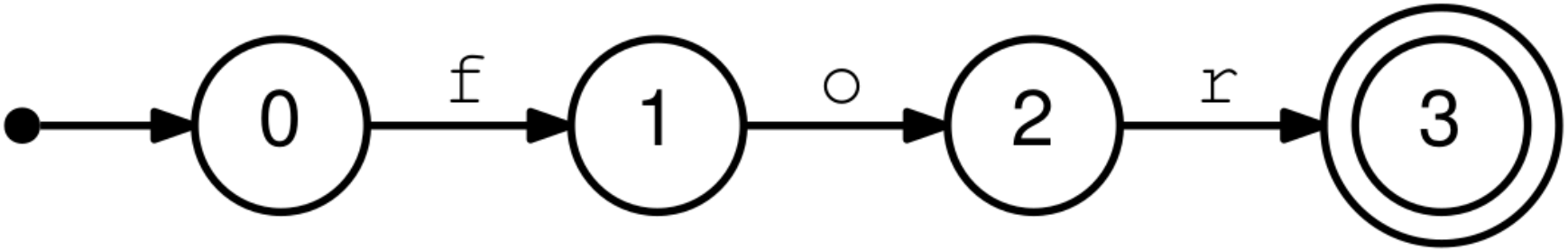


- This process to identify a lexeme parallels the behavior of a finite automaton (FA)
  - An FA has a set of states, some of which are known as accepting states (e.g., token defined)
  - If an FA ends up in an accepting state, the input is accepted (token emitted), otherwise rejected
  - This matches the concept of reading each character from the input (source code) and seeing if it is part of a lexeme or the final character of a lexeme (accepting state)

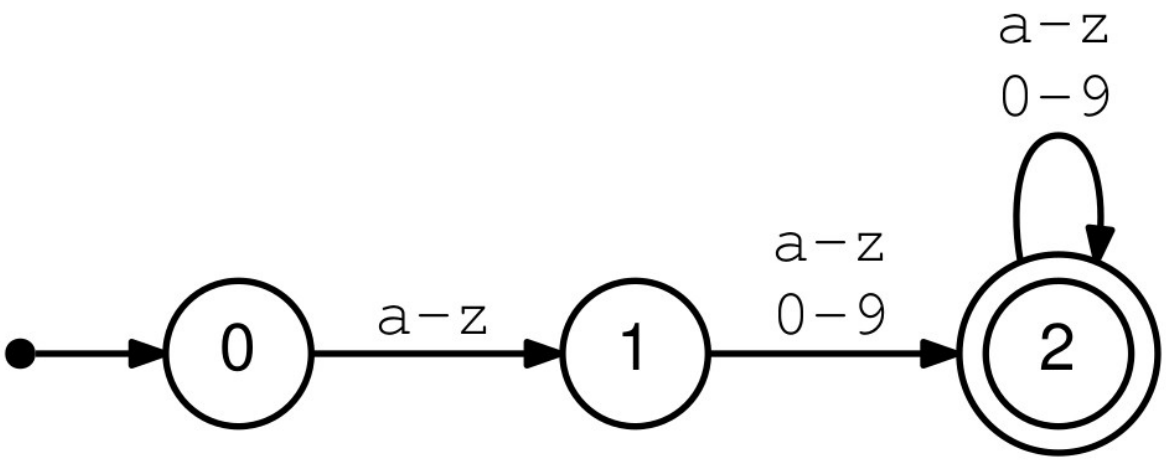
Deterministic Finite Automaton

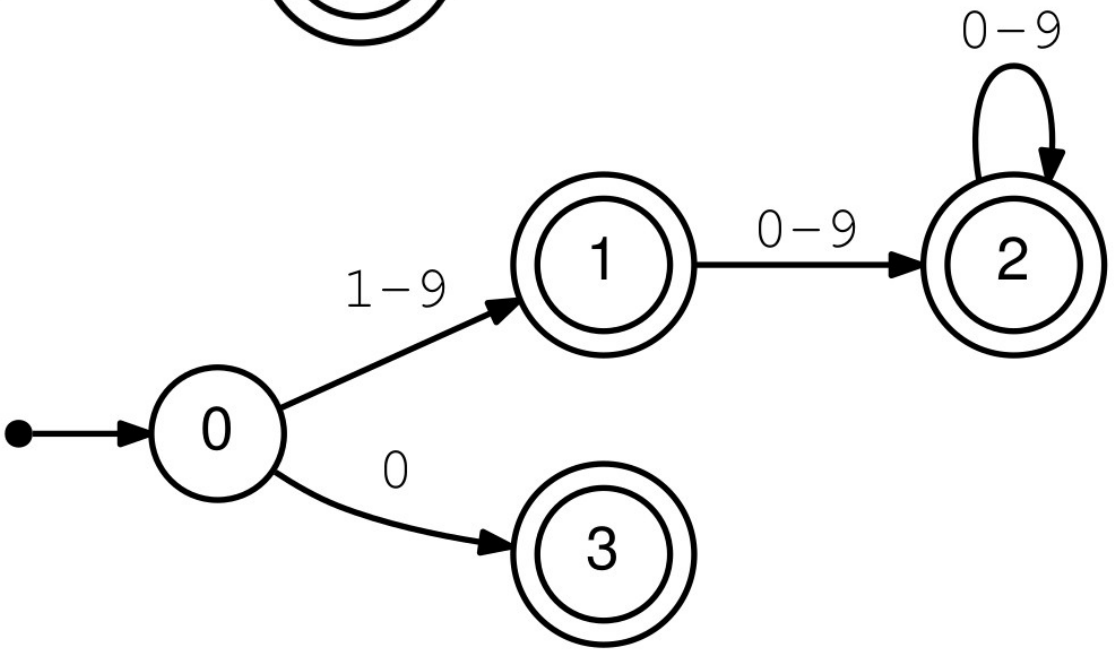


- A FA where there is no ambiguity in the state changes with respect to the inputs is known as a **Deterministic Finite Automaton** (DFA)

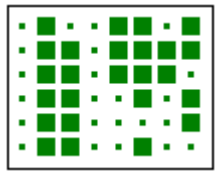


More DFA Examples

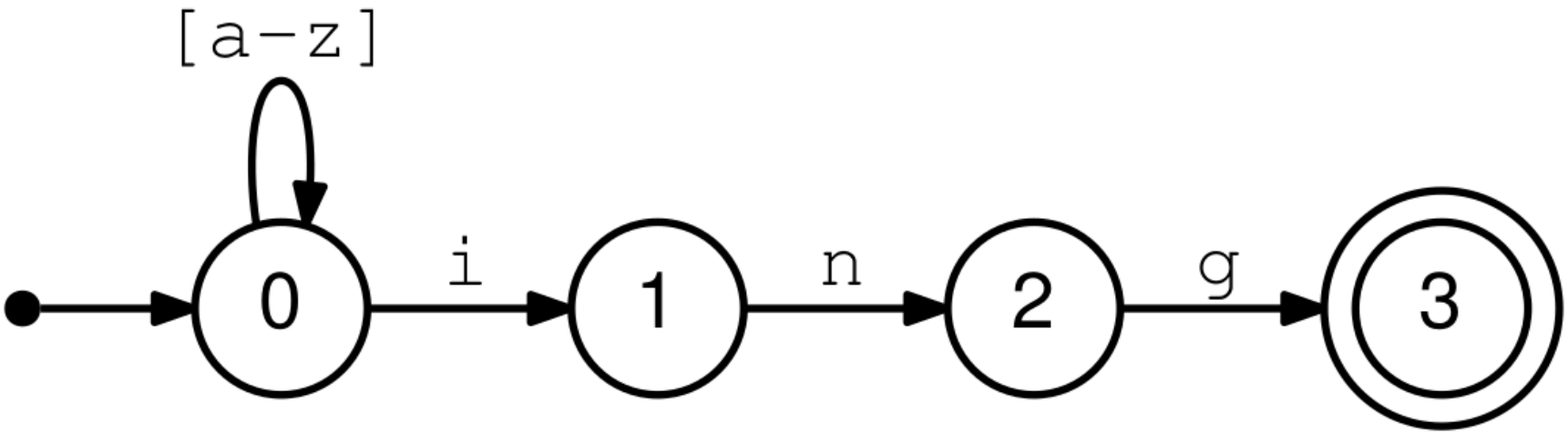




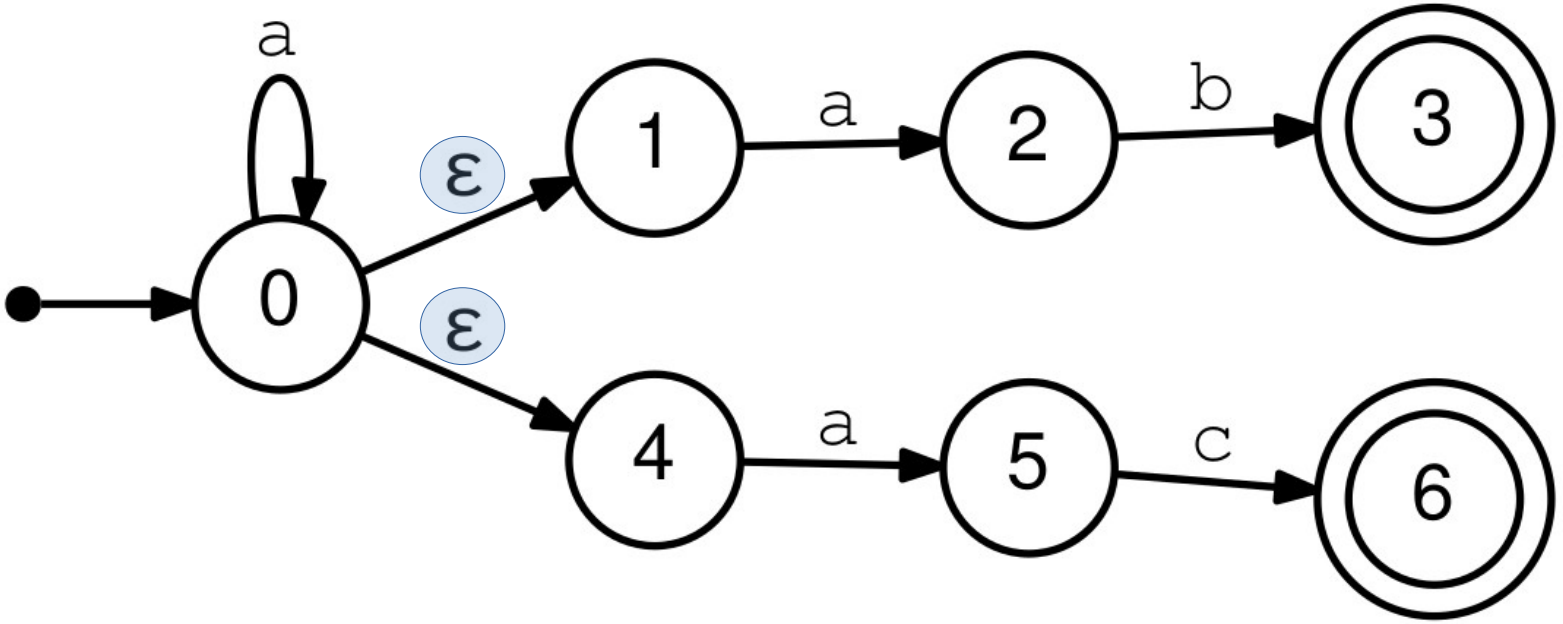
Nondeterministic Finite Automaton



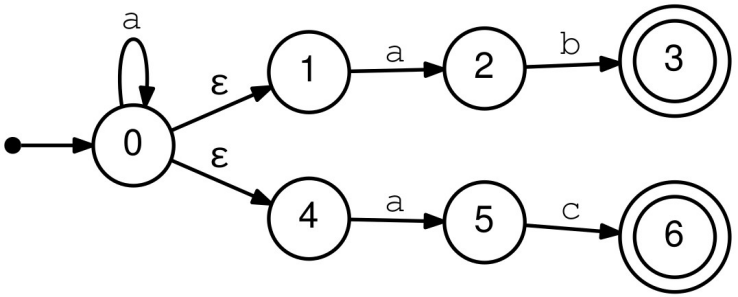
- A FA where there is ambiguity of the state changes with respect to the inputs is known as a **Nondeterministic Finite Automaton** (NFA)



Another NFA Example

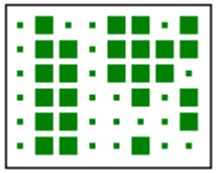


Many-worlds



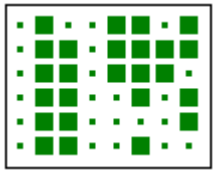
- Can picture NFA as being in multiple states at once when processing ambiguous input
- If **aaa** has been processed, NFA can be in state 0, 1, 2, 4, or 5. Without more input the ambiguity cannot be resolved (the multiverse!)
  - In this case **aaa** alone would be rejected
- If **c** is read after a set of **a**'s, the NFA can rule out states 1, 2, 3 (e.g. no **b**, now on a DFA path) and matches the last **a** to reach state 5 and **c** to reach state 6 → `accept`

NFA Can be Converted to a DFA



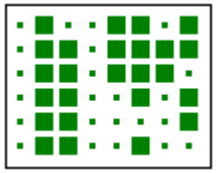
- We saw that once enough input was received the NFA reached a subset that was a DFA
- We can use that idea to convert an NFA into a DFA by accounting for each of the multiple worlds in the NFA and creating a DFA where each state is one of the NFA worlds
  - The process is discussed in the book and not required for us to deal with directly
- We now turn our attention to encoding our lexeme definitions

Formally Defining Lexemes



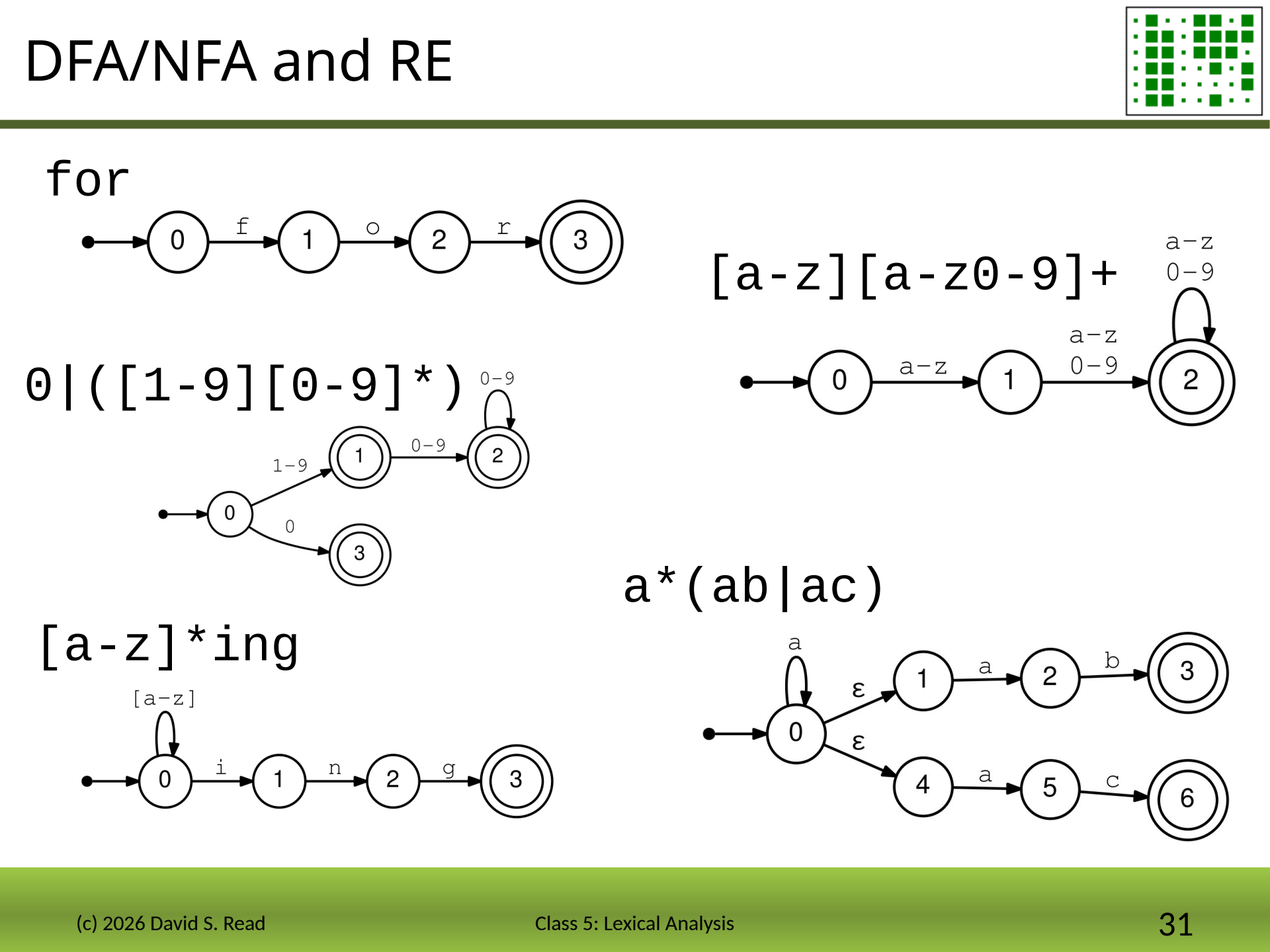
- Although a DFA is an efficient mechanism for processing our source code and identifying lexemes, we are left with a challenge of formally encoding the definition of the lexemes
  - We need a way to precisely define an NFA for each lexeme
  - Would like to automate the implementation of the NFA in a program
- **Regular Expressions** (RE) provide an approach

Regular Expressions

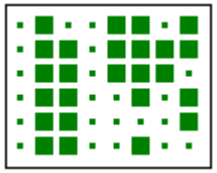


- Regular expressions (RE) are a syntax used to define patterns in text
  - We will spend time on RE syntax next week
- REs are easily converted to NFAs
  - There are several tools which do this. We'll use one, called **Flex**, for this purpose
- REs include *metacharacters* which represent the concepts we discussed in our DFA/NFA exploration
  - Repetition, ambiguities, options

DFA/NFA and RE

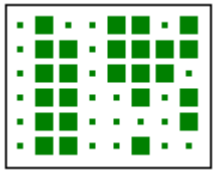


Process to Map Lexemes to Tokens



- Each token represents a concept in our programming language and is associated with a lexeme
  - The lexeme can be represented as a RE
- Our scanner consists of a set of tokens, each associated with a RE
- Only concern is resolving ambiguities between our REs
  - How can there be ambiguities between REs?

RE Ambiguity/Overlap Examples

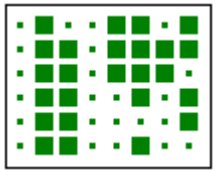


- If our language has **`NOT`** and **`NOT equals`** operators we might end up with these two token/RE mappings

Lexeme	Token
!	NOT
!=	NOT_EQUAL

- Why might this be an issue?
- How could this be addressed?
- Keywords? They match the RE for an id!

Decode a Java Statement (1/3)



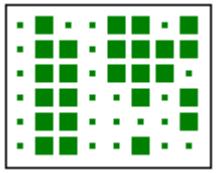
```
int index = 2 * count + 17;
```

Lexeme	Token
int	type
index	identifier
=	assign_op
2	int_literal
*	mult_op
count	identifier
+	plus_op
17	int_literal
;	semicolon

Parser Sees

```
type identifier assign_op int_literal mult_op
identifier plus_op int_literal semicolon
```

Decode a Java Statement (2/3)



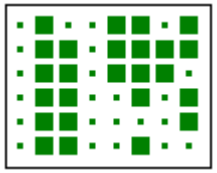
```
int numChar = (int)('Z' - 'A') + 1;
```

Parser Sees

```
type identifier
assign_op open_paren
type close_paren
open_paren char_literal
minus_op char_literal
close_paren plus_op int_literal
semicolon
```

Lexeme	Token
int	type
numChar	identifier
=	assign_op
(	open_paren
int	type
)	close_paren
(	open_paren
'Z'	char_literal
-	minus_op
'A'	char_literal
)	close_paren
+	plus_op
1	int_literal
;	semicolon

Decode a Java Statement (3/3)

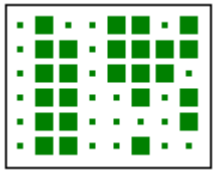


```
String fullName = "John" + ' ' + "Doe";
```

Lexeme	Token
String	type
fullName	identifier
=	assign_op
"John"	string_literal
+	plus_op
' '	char_literal
+	plus_op
"Doe"	string_literal
;	semicolon

Parser Sees

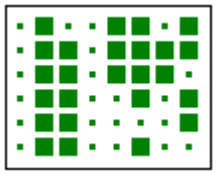
```
type identifier assign_op
string_literal plus_op
char_literal plus_op
string_literal semicolon
```



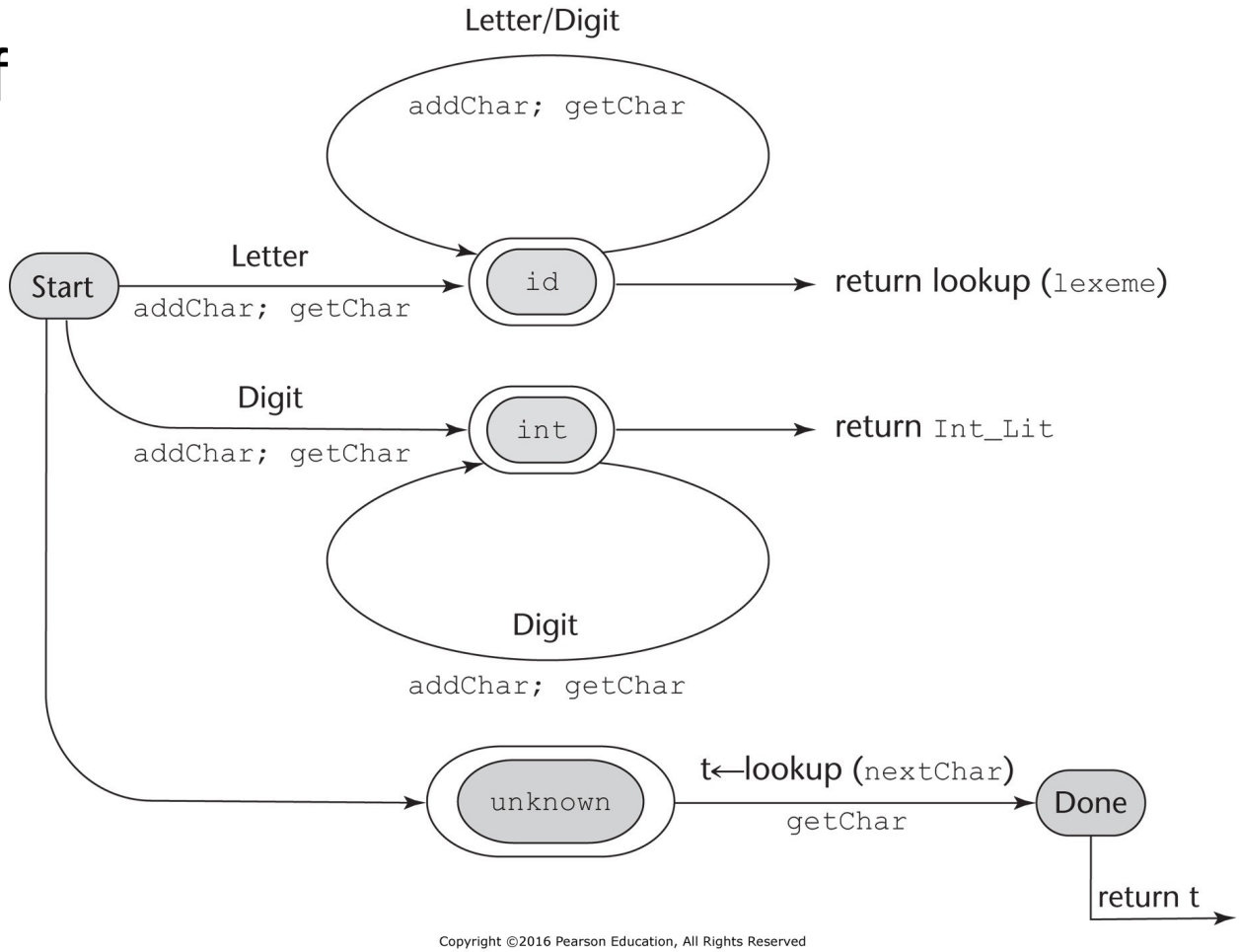
Lexical Analyzer Implementation Example

A Simple Scanner

Returning to a Prior State Diagram

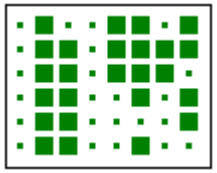


We'll look at an implementation of a lexical analyzer for this state diagram



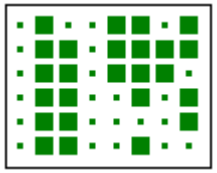
Copyright ©2016 Pearson Education, All Rights Reserved

Diagram to Lexical Analyzer Ex. (1/3)



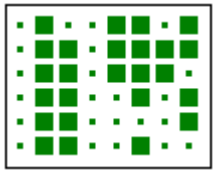
- Many lexemes resolve to the same token
  - Inefficient to define each of those lexemes separately
  - Consider, there are 52 letters and we don't want to have to define 52 patterns to match each letter as the start of a variable name (e.g. same token)
- Define a class of characters named LETTER ('a' - 'z', 'A' - 'Z') and transition on that class
- Same for digits, create class of characters named DIGIT ('0' - '9') and transition on it

Diagram to Lexical Analyzer Ex. (2/3)



- Need common operations and data for a lexical analyzer, use functions and variables to support the process
- `getChar()` - get the next character from the source file (skip whitespace) and identify the character class (e.g. LETTER, DIGIT)
- `nextChar` – character read by getChar()
- `charClass` – character class from getChar()
- `lookup()` - get the class of special characters (e.g. parentheses and arithmetic operators)

Diagram to Lexical Analyzer Ex. (3/3)

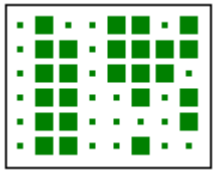


- We'll explore a C program that implements the lexical analyzer for this simple syntax
  - Want to see connection to concepts, not delve into the guts of C code
- Download `LexicalAnalyzerExample.tar.gz` from https://bits.monead.com/cs330/
- Move it to your home dir, expand it, and cd to it:

```
mv ~/Downloads/LexicalAnalyzerExample.tar.gz .
tar xvzf LexicalAnalyzerExample.tar.gz
cd LexicalAnalyzerExample
```

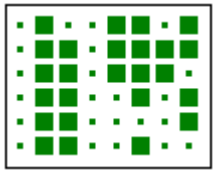
- Explore: `lexanalyzer.c` (open in a text editor)
- Compile: `cc lexanalyzer.c`
- Run: `./a.out expr1.txt`

`flex` – a lexical analyzer generator



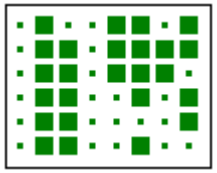
- Writing a lexical analyzer from a state diagram is straightforward but also redundant, with lots of boilerplate code
  - Operation of the analyzer is defined by the provided patterns and tokens
- Alternative to state diagram implementation is to use a regular language (e.g., regular expressions) and a corresponding lexical analyzer generator
  - The `flex` program has been used for many years to generate lexical analyzers based on regular expressions

Regular Language



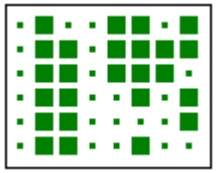
- To specify the parsing rules for tokens, `flex` supports the use of regular expressions along with some specialized syntax which defines the operation to occur when the regular expression is matched
  - `flex` reads its input file, which contains the patterns (REs) for tokens and generates a lexical analyzer library (C code implementing a set of DFAs)
- We incorporate the generated code as the scanner in our compiler

A flex Input File for the Example



```
%{
#include <stdio.h>
%}
%option main
%%
[0-9]*      printf("INT_LIT ");
[a-zA-Z]*   printf("IDENT ");
\+          printf("ADD_OP ");
\-          printf("SUB_OP ");
\*          printf("MULT_OP ");
\/          printf("DIV_OP ");
\(          printf("LEFT_PAREN ");
\)          printf("RIGHT_PAREN ");
\n          printf("\n");
[ \t]+      /* ignore whitespace */
%%
```

Using flex



- Run `flex` using the token definition file as input
  - Note there is a directive in the **`exv1.lex`** file telling flex to create a `main()` function – we will not use this feature for our real lexical analyzer

```
flex exv1.l
```

- Compile the generated lexical analyzer

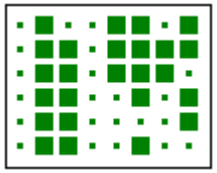
```
cc lex.yy.c
```

- Run the program and provide input to see the resulting tokens

```
./a.out < expr1.txt
```

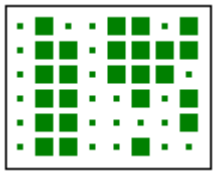
LEFT_PAREN IDENT ADD_OP INT_LIT RIGHT_PAREN DIV_OP IDENT

Formal Descript. vs. State Diagrams



- Formal Description → Generator
  - Using a tool like `flex` greatly reduces the effort involved in creating a lexical analyzer
  - Generated code is optimized for most cases
- State Diagram → Implementation
  - Diagram makes visualizing the patterns easier
  - Purpose built analyzer can be optimized for specific idiosyncrasies in a language

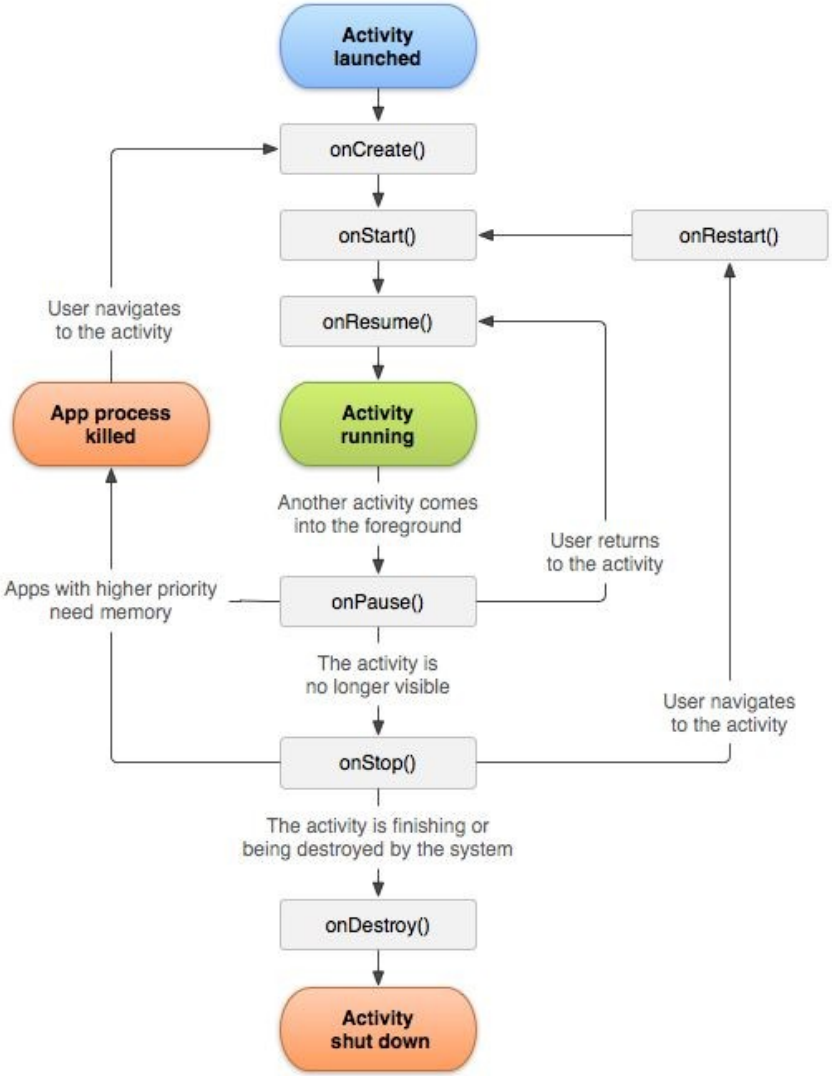
Example State Diagram for Android



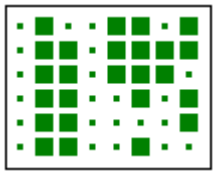
A brief aside…

State diagrams are used in a variety of ways. For example, here is a state diagram for an operating system, in this case the Android operating system's states for an application

Being able to read and write state diagrams is a useful skill since you will encounter them when working on applications. Further you may be asked to create them when designing and documenting systems (not just programming languages)



Homework



- Friday (9/18)
  - Make sure you can login to the Linux workstations
- Sunday (9/20):
  - Journal entry for week 2
    - **2-1 Rule** (minimum 2 paragraphs, 1 question)